PARSER

GTECH 3 - 2025/2026

CORRECTION

.OBJ

STRUCTURE D'UN .OBJ

https://en.wikipedia.org/wiki/Wavefront_.obj_file

v : information d'un vertex geométrique
vt: information d'un vertex textuel
vn: information d'un vertex normal
f: suite de vertice pour constituer une face: indiceV/indiceVt/indiceVn

Attention:
- L'ordre est important.
- Les indices commence a 1 et pas a 0.

EXERCICE

ON PARSE UN .OBJ

- Lire le fichier .obj
- Extraire les differentes informations de base: v, vt, vn, f
- Stocker les differentes informations correctement
- Stocker les faces ordonnées correctement

Pour aller plus regarder comment gérer plus d'arguments des .obj (Voir la page wikipedia)

Lien avec des .obj simple à lire:

https://groups.csail.mit.edu/graphics/classes/6.837/F03/models/
https://github.com/alecjacobson/common-3d-test-models/tree/master

OPENGL

INTRO

- Créez un projet vcpkg avec comme dépendance glfw3, glm et glad ou utilisez mon projet cmake. (si il marche sur les machines de l'école, j'espère vraiment, ...)
- Assurez vous que ca compile.

Définition Wikipedia:
OpenGL (Open Graphics Library) is a cross-language, cross-platform application programming interface (API) for rendering 2D and 3D vector graphics.

Vulkan tente d'être la suite de OpenGL.

GLFW

Glfw est une lib qui permet d'avoir une gestion de window, event, input, context opengl/vulkan.

Voici le code pour ouvrir une fenetre.

```cpp
#include <iostream>
#include <GLFW/glfw3.h>

int main() {
    glfwInit();
    GLFWwindow* win = glfwCreateWindow( width: 800, height: 600, title: "Basicgl", monitor: NULL, share: NULL);

    if (win == NULL)
    {
        const char* description;
        int code = glfwGetError(&description);
        std::cout << "Failed to create GLFW window: " << description << std::endl;
        glfwTerminate();
        exit(1);
    }

    while (!glfwWindowShouldClose(win)) {

        glfwSwapBuffers(win);
        glfwPollEvents();
    }

    return 0;
}
```

GLAD

Glad est un lib qui fait le lien entre les call openGL et votre programme.

```
glfwInit();
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, value: 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, value: 3);
glfwWindowHint(GLFW_OPENGL_PROFILE, value: GLFW_OPENGL_CORE_PROFILE);

GLFWwindow* win = glfwCreateWindow( width: 800, height: 600, title: "Basicgl", monitor: NULL, share: NULL);
```

<- On rajoute des informations de contexte.
Ici on utilise la version core 3.3 d'openGL

Ici on défini que le thread actuel est lié au context ->
On définit une callback qui sera appelée si le framebuffer change de taille.
Et on charge GLAD avec le context défini.

```
glfwMakeContextCurrent(win);
glfwSetFramebufferSizeCallback(win, FrameBufferSizeCallback);

if (!GLADloadproc(glfwGetProcAddress))
{
    std::cout << "Failed to initialize GLAD" << std::endl;
    exit(1);
}
```

Setup des shaders

LES SHADERS

Les shaders utilisent le langage GLSL: https://www.khronos.org/opengl/wiki/OpenGL_Shading_Language

Il y a deux types de shader:

- Le Vertex Shader, pour la géométrie
- Le Fragment Shader, pour le visuel.

```
const char* vertexShaderSource = "#version 330 core\n"
"layout (location = 0) in vec3 aPos;\n"
"void main()\n"
"{\n"
"    gl_Position = vec4(aPos.x, aPos.y, aPos.z, 1.0);\n"
"}\0";

const char* fragmentShaderSource = "#version 330 core\n"
"out vec4 FragColor;\n"
"void main()\n"
"{\n"
"    FragColor = vec4(1.0f, 0.5f, 0.2f, 1.0f);\n"
"}\n\0";
```

SETUP DES SHADERS

```
// Vertex shader creation => Only geometry
unsigned int vertexShader = glCreateShader(GL_VERTEX_SHADER);
glShaderSource(vertexShader, 1, &vertexShaderSource, NULL);
glCompileShader(vertexShader);

// Check if it works
int success;
char infoLog[512];
glGetShaderiv(vertexShader, GL_COMPILE_STATUS, &success);
if (!success) {
    glGetShaderInfoLog(vertexShader, 512, NULL, infoLog);
    std::cout << "VEXTEX SHADER : COMPILATION FAILED: \n" << infoLog << std::endl;
}
```

```
// Fragment shader creation => Only visual
unsigned int fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);
glShaderSource(fragmentShader, 1, &fragmentShaderSource, NULL);
glCompileShader(fragmentShader);

// Check if it works
glGetShaderiv(fragmentShader, GL_COMPILE_STATUS, &success);
if (!success) {
    glGetShaderInfoLog(fragmentShader, 512, NULL, infoLog);
    std::cout << "FRAGMENT SHADER : COMPILATION FAILED: \n" << infoLog << std::endl;
}
```

LE PROGRAM

```
// Attach shaders
unsigned int shaderProgram = glCreateProgram();
glAttachShader(shaderProgram, vertexShader);
glAttachShader(shaderProgram, fragmentShader);
glLinkProgram(shaderProgram);

// Check
glGetProgramiv(shaderProgram, GL_LINK_STATUS, &success);
if (!success) {
    glGetProgramInfoLog(shaderProgram, 512, NULL, infoLog);
    std::cout << "SHADER PROGRAM : LINK FAILED: \n" << infoLog << std::endl;
}
```

PREMIER CLEANUP

```
// Don't forget to delete shaders !
glDeleteShader(vertexShader);
glDeleteShader(fragmentShader);
```

Le triangle

LES VERTICES

```
// Points of triangle
float vertices[] = {
    -0.5f, -0.5f, 0.0f,
     0.5f, -0.5f, 0.0f,
    0.f, 0.5f, 0.0f,
};
```

```
//  Create new variable with uid in buffer
unsigned int VBO, VAO;
glGenVertexArrays(1, &VAO);
glGenBuffers(1, &VBO);
// Bind vertex buffer(s), and then configure vertex attributes
glBindVertexArray(VAO);

// specify variable type
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW); // GL_DYNAMIC_DRAW, GL_STREAM_DRAW

glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);

glBindBuffer(GL_ARRAY_BUFFER, 0);

glBindVertexArray(0);
```

LE RENDER

```
// Render
glClearColor(0.2f, 0.2f, 0.2f, 1.0f);
glClear(GL_COLOR_BUFFER_BIT);

//// Draw triangle
glUseProgram(shaderProgram);
glBindVertexArray(VAO);
glDrawArrays(GL_TRIANGLES, 0, 3);
glBindVertexArray(0);
```

LE CLEAN UP AVANT LA FIN

```
glDeleteVertexArrays(1, &VAO);
glDeleteBuffers(1, &VBO);
glDeleteProgram(shaderProgram);
```

EXERCICE

A VOUS DE JOUER

- Faire en sorte que ce qui a été vu dans les slides précédentes marche en local sur votre machine.
- Faire un Carré.
- Afficher les vertices venant d'un obj.